

2. a. Memory Efficient Doubly Linked List
–Creation of Empty List and Time Complexity

Program:

```
#include <stdint> // Required for uintptr_t (essential for
pointer XOR)
#include <stdlib.h> // Required for malloc and free
#include <iostream>

typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

Node *head;

Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}

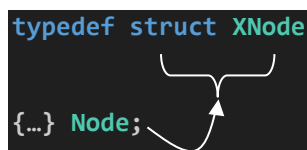
void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```

→ *Node*, is a structure variable i. e. the name given to struct *XNode* achieved through *typedef* keyword, gives new name to existing data types, here for struct *XNode*. Struct is a composite data type which contains group of several variables or different data types , grouped under a single name or single type.

Node is given name to struct *XNode* , As shown below:

```
typedef struct XNode
{
    ...
} Node;
```



We can also write it as:

```
typedef struct XNode
{
    int data;
    XNode *npx; // XOR of next and prev pointer
} Node;
```

And , now :

```
Node *head;
```

is a global variable, which are accessible to any part of the program, and as we need to start from head and keep track of it , its easy to perform by making the variable global otherwise with local scope we have to create each seperate head variable while calling each modular functions.

```

#include <cstdlib> // Required for uintptr_t (essential for
pointer XOR)
#include <stdlib.h> // Required for malloc and free
#include <iostream>

typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

Node *head;

```

The Rule: Zero – Initialization

In C ++, any variable declared at the global scope (outside of all functions) has static storage duration. The C ++ standard guarantees that variables with static storage duration are zero – initialized by the compiler if you do not explicitly initialize them.

For a pointer, "zero – initialization" means it is automatically set to a null pointer.

Node * head

<i>Address</i>	<i>Head</i>
0x021	<i>NULLPTR</i>

*And , then * head initialized to null pointer(explicitly).*

```
void createEmptyList()
{
    head = nullptr;
}
```



Note: It is just explicitly assigning `Node * head` to `nullptr`, which is already assigned with `NULLPTR` , since `Node * head` is global pointer variable, following :The Zero Initialization Rule.

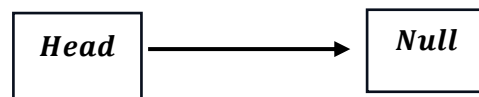
C + + version 11 → Null Pointer vs MACRO NULL

In C + +03 and earlier versions, there was no dedicated null pointer literal. Developers often used the constant 0 or macros like `NULL` to represent null pointers. The problem with this approach is that an integer value of 0 could accidentally be assigned to a pointer variable, leading to unexpected behavior and potential crashes.

In essence, `nullptr` is a special literal value that represents the absence of a valid pointer in C + +. It was introduced in the C + +11 standard to enhance type safety and improve code clarity. The `nullptr`, however, is a distinct type, `std :: nullptr_t`, designed explicitly for pointer absence. By using `nullptr`, the compiler can enforce type safety and distinguish between an intentionally null pointer and an integer value. This reduces the potential

for errors caused by accidental assignment of integers to pointers. The nullptr enhances code readability and self – documentation. Its explicit representation of a null pointer makes the code's intent clearer. Using nullptr makes the code more readable and less prone to errors. This helps prevent subtle bugs that could arise from accidentally assigning the wrong value to a pointer. The nullptr is a dedicated keyword that represents the null pointer value.

The nullptr is a keyword introduced in C++11 to represent a null pointer. It is used to indicate that a pointer does not point to any object or function. The nullptr has its own type, std::nullptr_t, which can be implicitly converted to any pointer type but not to integral types. This helps prevent errors where a null pointer might be confused with an integer. We can use nullptr wherever you need to assign or compare a pointer to a null value. Hence Creation of Empty Linked List , according to Linked List representation:



Head points to Null , is creation of empty linked list.

XORing Memory Address

```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
```

*In C++, `uintptr_t` stands for **Unsigned Integer Pointer Type**.*

It is a special integer type (defined in the `<stdint.h>` header) that is guaranteed to be **large enough to hold a memory address (pointer)** without losing data.

Here is the breakdown of why it exists and why it is critical for operations like the XOR Linked List.

1. The Problem: "How big is a pointer?"

In C++, the size of a pointer (`Node*`, `int*`, etc.) depends on your computer's architecture.

- **32-bit System:** A pointer is **4 bytes** (32 bits).
- **64-bit System:** A pointer is **8 bytes** (64 bits).

If you try to store a pointer inside a standard integer like `int`, you run a huge risk:

- An `int` is usually **4 bytes**.
- If you are on a 64-bit computer, your address is **8 bytes**.
- **Result:** If you cast the pointer to an `int`, you chop off half the address (truncation), causing the program to crash.

2. The Solution: `uintptr_t`

`uintptr_t` is a "smart" integer. The compiler automatically changes its size based on the system you are compiling for.

- *Compiling on 32-bit? `uintptr_t` becomes a 32-bit unsigned integer.*
- *Compiling on 64-bit? `uintptr_t` becomes a 64-bit unsigned integer.*

It guarantees: `sizeof(uintptr_t) == sizeof(void*)`

3. Visualizing the Safety

Imagine a 64 – bit Pointer Address: 0x7FFF5555 (Big number).

Scenario A: Using int (Dangerous)

Pointer (8 bytes): [0x7FFF5555]
int (4 bytes): [0x5555] < – – DATA LOSS! The '7FFF' is gone.

Scenario B: Using uintptr_t (Safe)

Pointer (8 bytes): [0x7FFF5555]
uintptr_t (8 bytes): [0x7FFF5555] < – – Perfect fit.

4. Why did we use it in the XOR Linked List?

In C ++, you are not allowed to use bitwise operators (like XOR ^) on pointers.

```
Node * a = ...;  
Node * b = ...;  
Node * c = a ^ b; // ERROR! Compiler forbids math on pointers.
```

To perform the math, we must:

- 1. Treat the pointers as just raw numbers (Integers).*
- 2. Perform the math.*
- 3. Turn the result back into a pointer.*

*Because we need an integer type that is **guaranteed** to hold the pointer safely, we cast to uintptr_t.*

// 1. Cast pointers to uintptr_t (Safe Integers)

```
uintptr_t addrA = (uintptr_t)a;  
uintptr_t addrB = (uintptr_t)b;
```

// 2. Perform Math

```
uintptr_t result = addrA ^ addrB;
```

// 3. Cast back to Pointer

```
return (Node *)result;
```

Summary Table

<i>Type</i>	<i>Size (Typical 64 – bit System)</i>	<i>Can hold a pointer?</i>	<i>Safe for XOR Logic?</i>
<i>int</i>	<i>4 bytes</i>	<i>No (Too small)</i>	<i>No</i>
<i>long long</i>	<i>8 bytes</i>	<i>Yes</i>	<i>Yes (but wastes space on 32 – bit systems)</i>
<i>uintptr_t</i>	<i>8 bytes</i>	<i>Yes (Perfect fit)</i>	<i>Yes (Portable)</i>

Using `long long` **might work accidentally on some systems**, but it is **NOT safe** and can **break your XOR linked list** depending on:

- CPU architecture (32-bit / 64-bit)
- Compiler
- Platform
- Pointer size

Let's break it down clearly.

1. Pointer size vs long long size

Case 1: 64-bit system

- Pointer size = 8 bytes (64 bits)
- `long long` = **also** 8 bytes (usually 64 bits)

Here, XOR might seem to work:

```
long long a = (long long)ptr; // allowed  
long long b = (long long)ptr2;  
long long x = a ^ b;  
Node * result = (Node *)x;
```

This will work on many 64-bit machines.

BUT not guaranteed by the C++ standard.

Case 2: 32-bit system

- *Pointer size = 4 bytes (32 bits)*
- *long long = 8 bytes*

*Now casting pointer → long long **extends** the pointer:*

Example pointer:

ptr = 0x8040ABCD

Cast to long long:

00000000 00000000 08040ABC D0000000 (64 – bit) // implementation dependent

Different compilers may fill the upper bits differently.

→ XOR breaks

→ Converted pointer becomes invalid

→ Program crashes while traversing

Case 3: Exotic architectures

On some systems:

- *Pointer = 128 bits (rare embedded or capability – based)*
- *long long = still 64 bits*

*Now pointer → long long is **guaranteed loss of data** → XOR list becomes corrupt.*

Case 4: Strict aliasing / alignment issues

Casting pointer → long long violates:

- **Pointer strict aliasing rules:**

The strict aliasing rule is a C/C++ optimization guideline that allows compilers to assume pointers of different types do not point to the same memory location. This rule enables aggressive optimization, but violating it (by accessing memory through a pointer of a different type, with exception → `char *`) can lead to undefined behavior and

incorrect program results. Programs must follow this rule unless specific compiler flags (like `-fno-strict-aliasing` in GCC) are used to disable the optimization, which can break other optimizations.

```
int foo(int * a, float * b) {  
    * a = 5;  
    * b = 3.14;  
    return * a; // < -- -- The Critical Line  
}
```

Strict Aliasing (The Fast Way):

1. Write 5 to address a.
2. Write 3.14 to address b.
3. Assume a and b cannot overlap because int and float are different types.
4. Return 5 immediately (no memory reload needed).

The "Universal Key" Exception (char*)

There is one major exception to this rule. `char*` (or `unsigned char*`) is allowed to alias anything. The standard acknowledges that sometimes need to look at the raw bytes of an object (like when copying memory or saving to a file).

```
int i = 5;  
  
char * ptr = (char *)&i; // This is LEGAL.  
  
cout << * ptr;          // This reads the first byte of i. Safe.
```

This is why functions like `memcpy` work perfectly – they cast everything to `char*` internally.

- **Alignment constraints** (*violates*):

"Violating alignment constraints" *refers to a hardware limitation regarding how the CPU accesses memory.*

When we use a long long (which is typically 8 bytes), the CPU expects it to be stored at a memory address that is a multiple of 8.

If we try to read or write a long long at an address like 0x1003 (which is not divisible by 8), we are violating alignment constraints.

The Rules of Alignment

Every data type has a "natural alignment" requirement:

Type	Size	Alignment Requirement	Allowed Addresses (Example)
Char	1 byte	Any address	0x01, 0x02, 0x03...
Int	4 bytes	Multiple of 4	0x00, 0x04, 0x08, 0x0C (INT = 12)...
long long	8 bytes	Multiple of 8	0x00, 0x08, 0x10 (INT = 16), 0x18 (INT = 24)...

Final Safe Rule

✓ `uintptr_t` is **the only correct type** for pointer arithmetic and XOR

✓ It is **legally guaranteed** to hold a pointer

✓ Works on **every architecture**

✓ No alignment problems

Why C + + created `uintptr_t`?

Because no other integer type is guaranteed to:

- be the same size as a pointer*
- be able to hold a pointer without losing bits*
- allow safe bitwise operations*

Even `size_t` is not guaranteed to match pointer size.

But `uintptr_t` is explicitly guaranteed to:

- ✓store pointer bit patterns*
- ✓XOR safely*
- ✓convert back safely*

Summary Table

<i>Type</i>	<i>Guaranteed to hold pointer bit?</i>	<i>Safe for XOR list?</i>
<i><code>uintptr_t</code></i>	<i>✓YES</i>	<i>✓SAFE</i>
<i><code>long long</code></i>	<i>✗NO</i>	<i>✗UNSAFE</i>
<i><code>unsigned long</code></i>	<i>✗NO</i>	<i>✗UNSAFE</i>
<i><code>size_t</code></i>	<i>✗NO</i>	<i>✗UNSAFE</i>
<i><code>unsigned int</code></i>	<i>✗NO</i>	<i>✗UNSAFE</i>
